# Code-Layout, Readability and Reusability

| | |
|---|---|
| **Date** | 14 July 2026 |
| **Team ID** | XXXXXX |
| **Project Name** | AgroSense AI – Intelligent Crop Recommendation System |
| **Maximum Marks** | 5 Marks |

## Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

## Code Layout Checklist:

| S.No | Code Quality Parameter | Description | Followed (Yes / No / Partial) | Remarks |
|---|---|---|---|---|
| 1 | Consistent Indentation | Uniform spacing/tabs used throughoutthe code | Yes | |
| 2 | Proper File Structure | Files and folders are logically organized | Yes | Proper indentation maintained throughout the project for better readability. |
| 3 | Meaningful Variable Names | Variables reflect their purpose clearly | Yes | Source files, templates, static assets, model, and dataset are organized logically. |
| 4 | Function / Method Names | Functions are descriptively named | Yes | Variables and functions use descriptive names representing their purpose. |
| 5 | Code Comments | Inline and block comments explain logc | Partial | Functions are clearly named according to their responsibilities. |
| 6 | Modular Design | Code is split into reusable functions/mdules | Yes | Important sections are documented; additional comments can improve maintainability. |
| 7 | No Redundant Code | Duplicate or unused code is removed | Yes | Duplicate logic is minimized to improve maintainability. |
| 8 | Error Handling | Exceptions and errors are handled gracefully | Partial | Basic validation and error handling are implemented; more comprehensive exception handling can be added. |

## Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology | Where Reused | Reusability Level (High / Medium / Low) |
|---|---|---|---|---|
| 1 | Input Validation Module | Python, Flask | User input processing | High |

| | | | | |
|---|---|---|---|---|
| 2 | Crop Prediction Module | Scikit-learn, Joblib | Machine learning prediction | High |
| 3 | Web Interface | HTML, CSS, JavaScript | User interaction pages | Medium |
| 4 | Dataset Loader | Pandas | Data preprocessing and model loading | Medium |
| 5 | Prediction Result Display | Flask Templates (HTML) | Displaying recommended crop | High |

**Overall Code Quality Assessment:**

| **Aspect** | **Rating (1-5)** | **Comments** |
|---|---|---|
| Code Layout & Structure | 5 | Project files are logically organized with a clear separation of frontend, backend, ML model, and dataset. |
| Readability | 4 | The code uses meaningful variable and function names with consistent formatting, making it easy to understand. |
| Reusability | 4 | The prediction logic and input validation can be reused in future agricultural applications or extended with new features. |
| Documentation / Comments | 3 | Basic documentation is available. Additional inline comments and developer documentation would further improve maintainability. |
| Overall Score | 4/5 | The project follows good coding practices, maintains a clean structure, and provides a reusable machine learning solution for crop recommendation. |